Database reference

projectAlp: Database Structure

MongoDB database `apar` — 20 collections, how they are shaped, and how they connect

Generated 12 Oct 2026 from the Mongoose models in `backend/src/models/` (branch `final`, commit 5dabd55). This describes the **schemas the code defines**. It was not produced by inspecting a live database, so a running database that predates a schema change may contain extra or missing fields.

---

1. Design at a glance and naming conventions
2. All 20 collections
3. Entity-relationship diagram
4. Core collections: users, faculty, faculty_profiles, departments, programmes, courses, students
5. IQAC record collections (type-discriminated): publications, patents, research_projects, phd_defences, faculty_activities, collaborations, student_activities, trainings, department_resources, teaching_methods
6. APAR collections: aparforms, iqac_approval_requests, notifications
7. Relationships (which field points where)
8. Indexes
9. Quirks and risks in the current design

1. Design at a glance

- **Database:** MongoDB 7, one database named `apar`. The code takes `MONGODB_URI` and appends `/apar`.
- **Access:** Mongoose 9 only. Every collection has one model in `backend/src/models/`; queries live in `backend/src/data-access/`.
- **No foreign keys.** MongoDB does not enforce relations and the schemas do not use Mongoose `ref`. Records point at each other with **plain string ids** (`faculty_id`, `department_id`, `student_id` …). Nothing stops a dangling id.
- **Business ids, not ObjectIds, are the real keys.** Every collection has its own unique string key such as `faculty_id` or `patent_id`. The Mongo `_id` exists but the application mostly uses the string key (it accepts either in lookups).
- **One collection, many resources.** The 37 IQAC data tables are stored in 10 collections. Each document carries a `type` field (`journal`, `fdp`, `mou` …) that says which resource it belongs to, and only that type's fields are filled.
- **Audit block.** Nearly every IQAC document has the same `metadata` sub-document (see below).

Id formats seen in the sample data

Id	Example	Meaning
`department_id`	CSE, ECE, ADMIN	Short department code. `ADMIN` is used for the IQAC Head, who has no real department.
`faculty_id / user_id`	FAC_PAWAN, IQAC001	A human-readable id. **`users.user_id` and `faculty.faculty_id` hold the same value for the same person.** That is the join between login accounts and staff records.
`paper_id, publication_id, …`	UUID v4	Generated in data-access with `uuid` when a record is created.
`ay (academic year)`	2026-27	Always stored short (`YYYY-YY`); a global middleware rewrites `2026-2027` to this form on the way in.

The shared `metadata` block

```
metadata: {
  created_at:  Date,          // set automatically (Mongoose timestamps map to these paths)
  updated_at:  Date,
  created_by:  String | null, // user_id of creator
  change_log: [ { action: 'created'|'updated'|'deleted',
                  user_id: String, timestamp: Date,
                  changes: Map(any)  // or plain String in the activity/publication models
                } ]
}
```

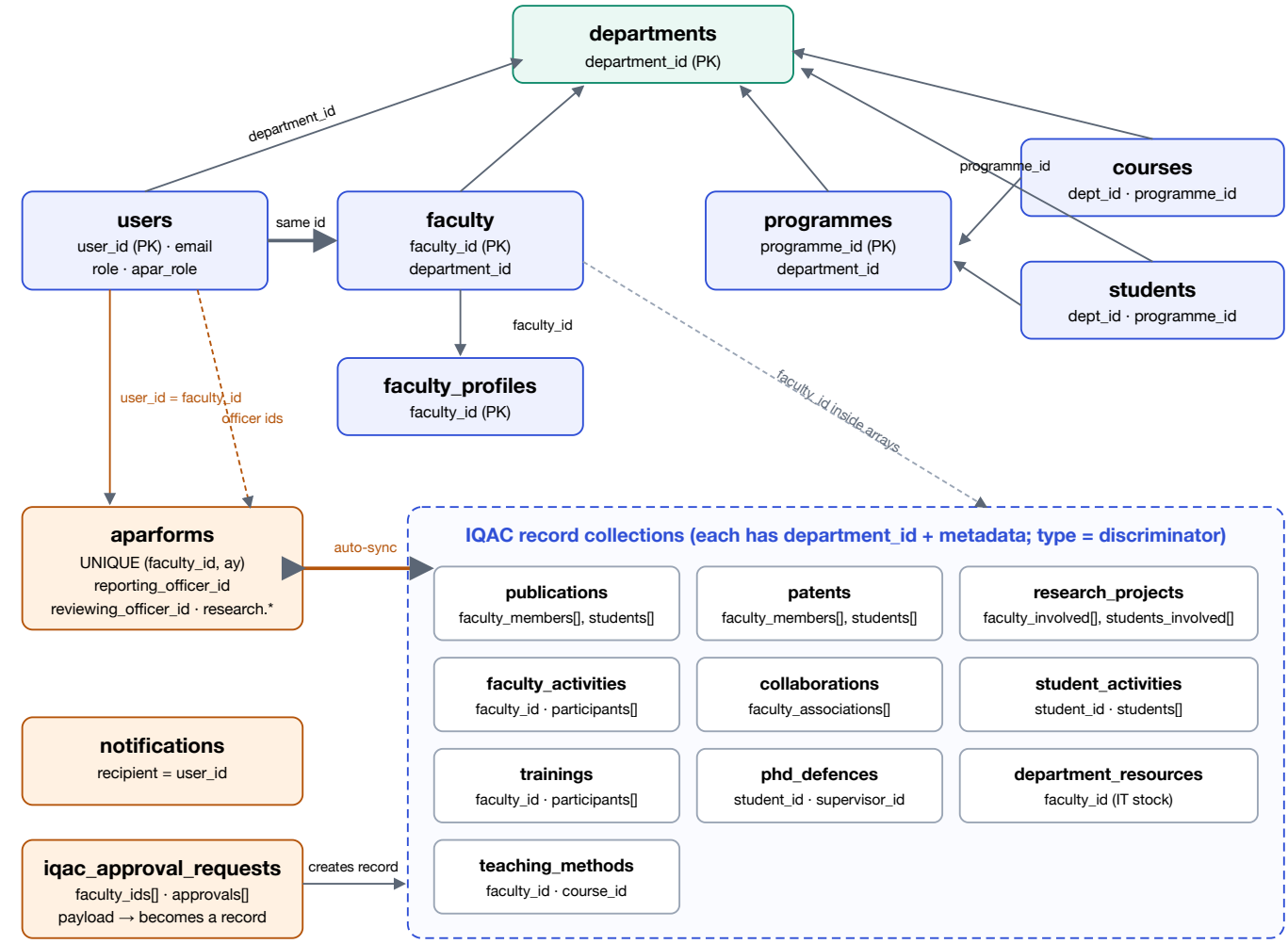
Used by: departments, courses, students, programmes, teaching_methods, faculty, patents, phd_defences, publications, research_projects, faculty_activities, collaborations, student_activities, trainings, department_resources. The change log feeds the "change history" and "audit log" modals in the UI. `users`, `faculty_profiles`, `aparforms`, `iqac_approval_requests` and `notifications` do not have it (they use `timestamps` or their own history).

2. All 20 collections

Collection	Model	Primary key (unique)	Holds
`users`	User	user_id, email	Login accounts, roles, session and lockout state
`faculty`	Faculty	faculty_id, email	Staff master record (name, designation, department)
`faculty_profiles`	FacultyProfile	faculty_id	Rich self-service profile with qualifications
`departments`	Department	department_id	Departments and their recognitions
`programmes`	Programme	programme_id	Degree programmes (type: standard, field_research)
`courses`	Course	course_id, course_code	Courses
`students`	Student	student_id	Student master record
`teaching_methods`	Teaching	method_id	Student-centric teaching methods per course
`publications`	Publication	publication_id	Journal, conference and book/chapter records
`patents`	Patent	patent_id	Patents
`research_projects`	ResearchProject	project_id	Funded projects, consultancy, corporate training
`phd_defences`	PhdDefence	defence_id	PhD supervision and defence
`faculty_activities`	FacultyActivity	activity_id	FDPs, visits, memberships, ICT use, awards, e-content, financial support
`collaborations`	Collaboration	collaboration_id	Collaborative activities, research exchange, MoUs, outreach
`student_activities`	StudentActivity	activity_id	Exams, higher education, performance, financial support
`trainings`	Training	training_id	Staff/professional training, mentoring, capability schemes
`department_resources`	DepartmentResource	resource_id	Library books, IT stock, professional schemes
`aparforms`	AparForm	(faculty_id, ay) pair	One APAR per faculty member per academic year
`iqac_approval_requests`	IqacApproval	_id	Pending faculty approvals for IQAC entries
`notifications`	Notification	_id	Per-user notifications

`aparforms` and `notifications` have no explicit `collection` option in their schema, so Mongoose derives the names (lower-cased, pluralised) from the model names. The other 18 set their collection name explicitly.

3. Entity-relationship diagram



Solid arrows mean “holds the id of”. Orange marks the APAR side. Every IQAC record collection also holds `department_id` (arrow to departments omitted for clarity). Students, courses and programmes are referenced by id from several IQAC collections (e.g. `phd_defences.student_id`, `teaching_methods.course_id`).

4. Core collections

users `collection: users`

Login identity. Authentication data lives here only; `faculty` does not store a password or role.

Field	Type	Notes
`user_id`	String	Required, unique, indexed. Equals `faculty.faculty_id` for staff.
`email`	String	Required, unique, lower-cased.
`name, designation`	String	Optional display data.
`password_hash`	String	Required. bcrypt.
`role`	enum	IQAC Head, Dean, Department HOD, Faculty Member (default). The **system role**.
`apar_role`	enum	Officer (Graded), Reviewing Officer, Reporting Officer, Dean. Optional.
`reporting_officer_id`	String	`user_id` of the assigned Reporting Officer.
`reviewing_officer_id`	String	`user_id` of the assigned Reviewing Officer.
`department_id`	String	Optional (IQAC Head uses `ADMIN`).
`session_token, session_expires_at`	String, Date	The currently valid JWT and its expiry; one session per user.
`must_change_password`	Boolean	Default false.
`failed_login_attempts, last_failed_login_at, locked_until`	Number, Date, Date	Lockout state.
`avatar_url`	String	MinIO object path of the profile photo.
`created_at, updated_at`	Date	Mongoose timestamps.

faculty `collection: faculty`

Field	Type	Notes
`faculty_id`	String	Required, unique.
`email`	String	Required, unique.
`name, designation`	String	Required.
`department_id`	String	Required, indexed.
`gender, phone, specialization, employment_type`	String	
`date_of_birth, joining_date`	Date	
`qualification, qualification_undergraduate, qualification_postgraduate, qualification_phd`	String	Copied into APAR Part I.
`metadata`	block	Standard audit block plus `migrated_from_sql_at` (left over from an earlier SQL-to-Mongo migration).

faculty_profiles `collection: faculty_profiles`

One document per faculty member (unique `faculty_id`), edited by the faculty member on the APAR profile page. Grouped sub-documents:

Group	Fields
`basic_info`	full_name, aadhaar_card, caste_category, profile_photo_url, gender, date_of_birth, nationality, marital_status, pan_card
`contact_info`	mobile_number, alternate_mobile_number, email_address, alternate_email_address, current_address, permanent_address (+ city, state, postal code for both), emergency_contact_name / number
`professional_info`	faculty_staff_id, designation, department, specialization[], date_of_joining, date_of_continuous_employment, employment_type, present_grade, years_of_experience, current_courses[], office_location, office_contact_number
`educational_qualifications[]`	Array of sub-documents with their own `_id`: degree, course, field_of_study, institution_name, university_board, year_of_passing, status, start_date, end_date, percentage_cgpa, certificate_url, exam_name, other_exam_name, exam_subject, conducting_org, year_of_qualification
`social_links`	linkedin_profile, personal_website, google_scholar_profile, researchgate_profile, orcid_id, scopus_author_id, vidwan_id

> **Sensitive data:** this collection stores Aadhaar and PAN numbers, addresses and phone numbers in plain text fields.

departments `collection: departments`

Field	Type	Notes
`department_id`	String	Required, unique (e.g. CSE).
`department_name, head_of_department, contact_email, contact_phone`	String	All required. `department_name` has a text index.
`location`	String	
`recognitions[]`	array	{recognition_id, name_of_agency, year_of_award, type, link}
`metadata`	block	

programmes `collection: programmes`

`type` = `standard` (default) or `field_research`.

Field	Type	Notes
programme_id	String	Required, unique.
department_id	String	Required, indexed.
programme_name	String	Required.
programme_code, level, duration_years, intake_capacity, academic_year	String / Number	Used by `standard`.
has_field_research_component, component_name, course_code_of_component, number_of_students_undertaking, link	mixed	Used by `field_research`.
metadata	block	

courses `collection: courses`

Field	Type	Notes
course_id	String	Required, unique.
course_code	String	Required, **unique across the whole collection**.
department_id	String	Required.
programme_id	String	Optional.
course_name, type, year_of_introduction	String	Required.
credits, semester_offered	Number	Required.
focus_on_employability, focus_on_entrepreneurship, focus_on_skill_development	Boolean	
metadata	block	

students `collection: students`

Field	Type	Notes
student_id	String	Required, unique.
department_id, programme_id	String	Required.
enrollment_no	String	Required, indexed (not unique).
name, email	String	Required.
level	enum	UG, PG, PHD, POST DOC, others.
gender, phone	String	
date_of_birth	Date	
year_of_admission	Number	Required.
current_semester	Number	
metadata	block	

teaching_methods `collection: teaching_methods`

Field	Type	Notes
method_id	String	Required, unique.
department_id, course_id, programme_id, faculty_id	String	All required.
method_type, level, academic_year, semester, details_of_methods	String	Required.
course_name, programme_name	String	Required; **denormalised copies** of the course/programme names.
course_code, assessment_method, outcome, remarks, evidence_link	String	
metadata	block	

5. IQAC record collections

These hold the data entered on the IQAC portal. They share one pattern: a unique business key, a required `department_id`, a `type` discriminator, a flat bag of fields for *all* the types, array fields naming the people involved, and the `metadata` block. For brevity, the type-specific fields are grouped under the resource they belong to; fields not in that group are simply absent (`undefined`) on that document.

publications `collection: publications` | `key: publication_id`

`type` ∈ `journal`, `conference`, `book` (required). For journals the code sets `paper_id` to the same UUID as `publication_id`.

Common: `department_id (req)` `title` `year_of_publication (String)` `academic_year` `doi` `indexing` `link`
journal:
`paper_id` `name_of_journal` `author_names` `issn` `volume` `issue` `page_numbers` `impact_factor` `citation_count (Number)` `is_ugc_care_listed (Boolean)` `link_to_paper`
conference: `name_of_conference` `conference_level` `organizer` `venue` `publisher` `paper_status` `isbn` `award_received`
book:
`publication_type (Book|Chapter)` `title_of_book` `title_of_chapter` `role` `year` `isbn_number` `name_of_publisher` `publisher_type` `same_institute_affiliation (Boole`
People:
`faculty_members[{faculty_id, author_type, name, emp_code, role}]` `faculty_ids[String]` `students[{student_id, name, roll_no}]` `student_ids[String]` `external_a`

patents `collection: patents` | `key: patent_id`

`department_id (req)` `patent_title (req)` `author_names` `application_number (req)` `patent_number` `status (req: Filed|Published|Granted)` `country` `centres (req)`
People (untyped arrays): `faculty_members[]` `students[]` `external_inventors[]`

research_projects `collection: research_projects` | `key: project_id`

`type` ∈ `funding`, `consultancy`, `corporate_training`.

Common: `department_id (req)` `title` `role` `start_date` `end_date` `status` `outcome` `remarks` `link` `academic_year`
funding: `funding_id` `title_research` `type_of_project` `funding_agency_name` `funding_type` `sanction_number` `year_of_sanction` `amount (Number)` `chair_holder`
consultancy:
`consultancy_id` `name_of_project` `agency_name` `type_of_agency` `consultancy_type` `grant_amount` `revenue_generated` `duration_start_date` `year_of_consultancy`
corporate_training:
`training_id` `name_of_program` `mode_of_training` `no_of_participants` `year_of_training` `agency_contact_details` `number_of_faculties` `number_of_students` `number_`
People:
`faculty_involved[]` `students_involved[]` `external_collaborators[]` `external_consultants[]` `external_trainers[{name,email,role,affiliation}]` `manpower_details`

phd_defences `collection: phd_defences` | `key: defence_id`

`student_id (req)` `department_id (req)` `enrollment_no (req)` `student_name (req)` `thesis_title (req)` `thesis_type (req)` `supervisor_id (req)` `supervisor_name (r`
Arrays:
`co_supervisors[{affiliation_type: Internal|External, co_supervisor_id, external_name, external_affiliation}]` `committee_members[{member_id, role}]` `externa`

faculty_activities `collection: faculty_activities` | `key: activity_id`

`type` ∈ fdp, visit, affiliation, ict, award, econtent, financial_support. Optional `faculty_id` (indexed) for single-person records.

Common: `department_id (req)` `faculty_id` `academic_year` `start_date` `end_date` `outcome` `remarks` `link`
fdp:
`program_id` `program_title` `type_of_program` `level` `participation_level` `amount_for_funding` `mode` `duration_days` `organising_body` `funding_agency` `venue` `certifi`
visit: `visit_id` `organisation_name` `title` `visit_type` `purpose` `location`
affiliation: `affiliation_id` `professional_body_name` `membership_id` `type_of_membership` `position_held` `area_of_support` `status`
ict:
`ict_id` `course_name` `semester` `ict_mode` `ict_tools_used[String]` `faculty_name` `ict_classroom_rooms[String]` `e_resources_and_techniques_used` `no_of_ict_enabled_`
award:
`award_id` `name_of_award` `type_of_award` `category_of_award` `name_of_organisation` `awarding_agency` `agency_type` `date_of_award` `monetary_value` `year` `faculty_rec`
econtent:
`econtent_id` `course_id` `course_code` `name_of_module` `type_of_content` `platform` `platform_type` `date_of_launching` `target_audience` `duration_hours` `learning_out`
financial_support: `support_id` `title_of_event` `event_type` `host_institution` `amount` `date_start` `date_end`

collaborations `collection: collaborations` | `key: collaboration_id`

`type` ∈ activity, exchange, mou, outreach.

Common: `department_id (req)` `title` `level` `start_date` `end_date` `academic_year` `outcome` `remarks` `link`
activity:
`activity_id` `title_of_activity` `name_of_collaborative_agency` `type_of_activity` `other_type_of_activity` `number_of_participants` `source_of_financial_support` `f`
exchange: `year` `duration` `nature_of_activity` `name_of_institution` `funding_agency` `nature_of_collaboration`
mou: `mou_id` `organisation_name` `type_of_mou` `year_of_signing` `purpose` `activities_under_mou (Mixed)`
outreach: `location` `geo_tag_location_link` `no_of_participants` `target_beneficiaries` `sponsoring_agency`
People: `faculty_associations[]` `student_associations[]` `external_contributors[]`

student_activities `collection: student_activities` | `key: activity_id`

`type` ∈ exam, higher_ed, performance, financial_support. Optional `student_id` (indexed).

Common: `department_id (req)` `student_id` `student_name` `academic_year` `remarks` `link`
exam: `exam_record_id` `name_of_exam` `type_of_exam` `level_of_exam` `year_of_qualifying` `rank_or_score` `attempt_number` `programme_applied_for` `result_status`
higher_ed: `record_id` `institution_joined` `country` `level_of_study` `programme_type` `details_of_programme` `mentor_name` `year_of_joining` `current_status`
performance:
`performance_id` `name_of_award` `type_of_activity` `level_of_award` `category` `position_or_rank` `organizing_body` `year` `outcome` `students[{name, roll_no, student_a`
financial_support:
`support_id` `name_of_event` `type_of_support` `funding_agency` `amount_of_support` `date_of_event` `students_supported[{student_id, student_name, amount, pan_no}]`

trainings `collection: trainings` | `key: training_id`

`type` ∈ staff, professional, mentoring, capability.

Common: `department_id (req)` `title` `type_of_training` `mode` `semester` `start_date` `end_date` `academic_year` `outcome` `remarks` `link`
staff: `training_record_id` `name_of_official` `program_name` `organising_agency` `funding_details` `participants[{faculty_id, role}]`
professional:
`title_of_event` `year_of_training` `number_of_participants` `sponsoring_agencies` `faculty_participants[{faculty_id, role}]` `student_participants[{student_id, ro`
mentoring:
`mentor_record_id` `faculty_id` `activity_type` `mentor_mentee_ratio` `target_group` `date_of_activity` `duration_hours` `organizer_name` `mentor_name` `student_details`
capability:
`scheme_id` `name_of_scheme` `type_of_scheme` `no_of_students_enrolled` `name_of_agencies_involved` `faculty_ids[{faculty_id, role}]` `students[{name, roll_no, role`

department_resources `collection: department_resources` | `key: resource_id`

`type` ∈ library, it_stock, scheme.

Common: `department_id (req)` `remarks` `link`
library: `book_id` `title_of_book` `author` `publisher` `year_of_publication` `isbn_number` `edition` `type_of_book` `no_of_copies` `no_of_students_using`
it_stock:
`stock_id` `lab_name` `faculty_id` `no_of_desktops` `no_of_servers` `no_of_workstations` `no_of_hpcs` `total_storage_tb` `internet_bandwidth_mbps` `software_list` `total_`
scheme:
`scheme_id` `name_of_scheme` `type_of_scheme` `name_of_organisation` `funding_agency` `sanction_number` `principal_investigator` `co_investigators` `year_of_sanction` `a`

6. APAR collections

aparforms `collection: aparforms (derived name)` | `unique (faculty_id, ay)`

One document per faculty member per academic year. The document is a large nested structure that mirrors the six printed parts of the form.

```
aparforms
├─ faculty_id (req, idx)   ay (req, idx)   status (default 'Draft')   current_step (default 1)
├─ reporting_officer_id, reporting_date, reviewing_officer_id, reviewing_date
├─ query_comment, reporting_query, reviewing_query          (latest query text)
├─ monthly_saved_at                                        (set by "Save as Monthly"; required before submit)
├─ timeline { submitted_at, reporting_reviewed_at, reviewing_reviewed_at }
├─ history [ { action, by, date, comment } ]               (every state change)
│
├─ personal   (Part I)    name, designation, date_of_birth, email, phone, department_id,
│                         qualification_undergraduate / postgraduate / phd, joining_date,
│                         report_start_date, report_end_date, section_officer, sc_st_status,
│                         absence_taken, absence_period, title, academics, caste, grade
│
├─ teaching   (Part II)   health_checkup_file
│                         description_of_duties_department[ {description, proof} ]
│                         description_of_duties_admin[ {description, proof} ]
│                         courses_taught[ {name_of_course, course_code, total_lectures_scheduled/engaged,
│                              extra_lectures_engaged, tutorials_*, labs_*, reasons_not_engaged,
│                              degree_type, semester, course_type} ]
│                         time_table{provided,actual}  workload_week{odd,even semester}
│                         teaching_methods[], faculty_visits_other_institutions[], ict_tools[],
│                         student_centric_methods[], tutorials_tests[], academic_planning[] ...
│
├─ research   (Part III)  journals[]  conferences[]  books[]  projects[]  consultancy[]  patents[]
│                         awards[]  e_content[]  faculty_visits[]  collaborations[]  phd_supervision[]
│                         mous[]  workshops[]   + free-text lists (memberships_text, other_activities,
│                         summer_institutes_*, ug_pg_guidance, research_guidance, industry_interaction)
│
├─ corporate  (Part IV)   curriculum_development[], course_development_details[], lab_development[],
│                         cultural_activities[], sports_community[], admin_assignment_university /
│                         department / external[], any_other[]   (each [ {description} ]),  certify
│
├─ assessment (Part V, Reporting Officer)
│                         section_a{q1..q4, overall_grading}  section_b{q1..q11 (+q7a,q7b), overall_grading}
│                         section_c{q1..q6, overall_grading}   general{q1..q6}
│
└─ remarks    (Part VI, Reviewing Officer)
                          length_of_service, satisfied_with_reporting, agree_with_assessment,
                          disagreement_reason, general_remarks, specific_characteristics,
                          signature_place, signature_date, query_comment
```

status values

Free-form string, no enum in the schema. Values the code writes: `Draft`, `Submitted`, `Forwarded by Reporting officer`, `Query Raised by Reporting officer`, `Accepted by Reviewing officer`, `Query Raised by Reviewing officer`. Older values (`Query Raised`, `Verified`, `Reviewed`) still appear in code and in the dashboard.

How `research.*` rows map back to IQAC collections

research key	IQAC collection / type	Key field in the row
`journals`	publications / journal	paper_id
`conferences`	publications / conference	paper_id
`books`	publications / book	publication_id
`projects`	research_projects / funding	project_id
`consultancy`	research_projects / consultancy	consultancy_id
`patents`	patents	patent_id
`awards`	faculty_activities / award	award_id
`e_content`	faculty_activities / econtent	econtent_id
`faculty_visits`	faculty_activities / visit	visit_id
`workshops (fdps)`	faculty_activities / fdp	program_id (delete propagation uses activity_id)
`collaborations`	collaborations / activity	activity_id
`mous`	collaborations / mou	mou_id
`phd_supervision`	phd_defences	defence_id

The APAR form holds a **copy** of each row. The copy and the IQAC record are linked only by that id string; the sync code copies in both directions.

iqac_approval_requests `collection: iqac_approval_requests`

Field	Type	Notes
`resource_id`	String	Required, indexed. Which IQAC resource this will become (e.g. `journal_research_papers`); looked up in `iqacApprovalRegistry.js`.
`resource_title, title`	String	Display names.
`payload`	Mixed	The full form data waiting to be saved. Faculty may replace it with a corrected version when approving.
`faculty_ids[]`	String[]	Everyone who must approve (extracted from the payload).

`approvals[]`	array	{faculty_id, status: pending|approved|rejected, comment, acted_at}
`status`	enum	pending, approved, rejected, failed. Indexed.
`created_by`	sub-doc	{user_id, name, role, department_id}
`permanent_record_id, finalized_at, finalization_error`	String, Date, String	Set once all approve (id of the record created) or when creation fails.
`createdAt, updatedAt`	Date	Mongoose timestamps.

A rejection deletes the document outright; only pending, approved and failed requests persist.

notifications `collection: notifications (derived name)`

Field	Type	Notes
`recipient`	String	Required, indexed. A `user_id`, or a role bucket such as `IQAC_HEAD` / `HOD` / `ADMIN`.
`sender`	String	Default `System`.
`type`	enum	APAR_SUBMISSION, APAR_REVIEW_REQUEST, APAR_COMPLETED, IQAC_UPDATE, IQAC_APPROVAL_REQUEST, IQAC_APPROVAL_DECISION, SYSTEM_ALERT, CROSS_FACULTY_REMOVAL
`title, message`	String	Required.
`link`	String	Front-end route to open, e.g. `/apar-form`.
`isRead`	Boolean	Default false.
`metadata`	Mixed	Entry type, display type, entry id, submitter, academic year, action.
`createdAt, updatedAt`	Date	Declared explicitly *and* added by `timestamps: true`.

7. Relationships: which field points where

From
`users.user_id`
`faculty.faculty_id`
`users.reporting_officer_id, reviewing_officer_id`
`*.department_id (users, faculty, programmes, courses, students and all IQAC collections)`
`courses.programme_id, students.programme_id, teaching_methods.programme_id`
`teaching_methods.course_id, faculty_activities.course_id (econtent)`
`teaching_methods.faculty_id, faculty_activities.faculty_id, trainings.faculty_id, department_resources.faculty_id`
`publications.faculty_members[].faculty_id, patents.faculty_members[], research_projects.faculty_involved[], collaborations.faculty_associations[], faculty`
`publications.students[].student_id, patents.students[], phd_defences.student_id, student_activities.student_id`
`phd_defences.supervisor_id, co_supervisors[].co_supervisor_id, committee_members[].member_id`
`aparforms.faculty_id`
`aparforms.reporting_officer_id, reviewing_officer_id`
`aparforms.research.*[ id ]`
`iqac_approval_requests.faculty_ids[], approvals[].faculty_id`
`iqac_approval_requests.permanent_record_id`
`notifications.recipient`
`*.metadata.created_by, change_log[].user_id`

8. Indexes

Unique indexes are what keep ids from repeating; compound indexes speed up the dashboard and list endpoints, which almost always filter by `type` plus department, year or status.

Collection	Unique	Other indexes
`users`	user_id, email	session_token
`faculty`	faculty_id, email	department_id; text(name, email)
`faculty_profiles`	faculty_id	—
`departments`	department_id	text(department_name)
`programmes`	programme_id	department_id; type; (type, department_id)
`courses`	course_id, course_code	department_id; (department_id, programme_id)
`students`	student_id	department_id; enrollment_no; (department_id, programme_id)
`teaching_methods`	method_id	department_id; faculty_id; (department_id, academic_year); (faculty_id, course_id)
`publications`	publication_id	department_id; type; (type, department_id); (type, year_of_publication desc); faculty_members.faculty_id
`patents`	patent_id	department_id; (department_id, status); faculty_members.faculty_id
`research_projects`	project_id	department_id; type; (type, department_id); (type, status); faculty_involved.faculty_id
`phd_defences`	defence_id	student_id; department_id; supervisor_id; (department_id, academic_year)

faculty_activities	activity_id	faculty_id; department_id; type; (type, faculty_id); (type, department_id); (type, academic_year)
collaborations	collaboration_id	department_id; type; (type, department_id); (type, academic_year); faculty_associations.faculty_id
student_activities	activity_id	student_id; department_id; type; (type, student_id); (type, department_id); (type, academic_year)
trainings	training_id	department_id; type; (type, department_id); (type, academic_year); faculty_id
department_resources	resource_id	department_id; type; (type, department_id)
aparforms	(faculty_id, ay)	faculty_id; ay
iqac_approval_requests	—	resource_id; status; (faculty_ids, status); (created_by.user_id, createdAt desc)
notifications	—	recipient; (recipient, isRead)

9. Quirks and risks in the current design

Observed from the schemas. Nothing was changed.

Integrity

- **No referential integrity.** Deleting a department, faculty member or student leaves their ids behind in every collection that mentions them. Nothing cascades.
- **Two copies of research data.** Each research row exists in an IQAC collection and inside `aparforms.research`. They are kept aligned by application code; a failed sync or a direct edit in the database leaves them different with no automatic repair.
- **Unique `course_code` is global.** Two departments cannot use the same course code even for different programmes.
- **Person arrays are untyped.** `patents.faculty_members`, `research_projects.faculty_involved`, `faculty_activities.faculty_participants`, `collaborations.faculty_associations` are declared as bare `[]`, so Mongoose accepts any shape; only the code convention (`{faculty_id, ...}`) keeps them usable. Publications, by contrast, define the sub-document fields.
- **`status` in `aparforms` is an open string** (no enum), so a typo creates a new, unrecognised state. Several older values are still referenced.

Shape and typing

- **Wide, mostly-empty documents.** A `faculty_activities` document of type `award` has the schema's ~100 other fields unused. This is the price of one collection per family.
- **Many "alternate key" fields.** Each type keeps its historical id (`program_id`, `visit_id`, `mou_id` …) beside the shared primary key (`activity_id`, `collaboration_id` …). The APAR form uses the alternate ones, while some propagated deletes use the primary one, so the two must hold the same value for the match to work.
- **String vs Number.** The APAR form stores numeric values as strings (amounts, years, counts) "to be safe for drafts", whereas IQAC collections use `Number` for many of the same fields (the schema comments say "IQAC: Number"). `publications.year_of_publication` is a String, yet `update()` and `deletePaper()` in the journal data-access compute `year_of_publication + 1`, which concatenates (`"2024"+1` gives `"20241"`). It is harmless today only because the auto-sync trigger ignores the year it receives.
- **`change_log.changes` has two types:** a `Map` in the core models, a plain `String` in the activity/publication models.
- **Denormalised names** (`course_name`, `programme_name`, `student_name`, `supervisor_name`, `faculty_name`) are stored beside ids and will not update if the source changes.
- **Academic year is a String in two formats** (`2026–27` canonical, `2026–2027` historical). The code queries both variants when syncing.
- **Mixed id vocabularies.** Some code paths receive a Mongo ObjectId, others a `user_id` or `faculty_id`; `resolveToReadableId` converts between them.

What is solid

- Every collection has a unique business key, and all list/filter paths used by the UI have a matching index.
- `aparforms` cannot be duplicated: the unique (`faculty_id`, `ay`) index also protects against the race of two simultaneous first loads (the code handles the resulting duplicate-key error).
- Sessions and lockouts are stored on the user document, so they survive restarts and work across several backend instances.
- The audit `metadata.change_log` gives per-record history without a separate audit collection.

End of document.